# Lab 5

Start Assignment

- Due Feb 27 by 11:59pm
- Points 100
- Submitting a file upload
- Available after Feb 18 at 12am

# Overview

In this lab, we will focus on how to generate smooth trajectories while following a path with specified waypoints. We will implement two different approaches:

**1) Gradient Descent Path Smoothing.** The task is to apply gradient descent method to minimize a cost function that balances two cost terms:

- Path Following: Keeping the new (smoothed) path close to the original waypoints.
- Path Smoothing: Ensuring that the path changes gradually between neighboring points.

**2) Third-Order Polynomial Time Scaling.** The task is to implement the 3rd order polynomial time scaling and apply it for each segment of the trajectory and for both x and y coordinates. Waypoints will be provided in the sample script, and should be included in the boundary/continuity constraints when generating trajectories.

# Sample Code for Polynomial Time Scaling

- Open a new terminal and go to your `eecs256aw25` package. We will start from a new python script.

```
cd ~/ros2_ws/src/eecs256aw25/eecs256aw25/
touch gradient_descent.py
gedit gradient_descent.py
```

- Copy and paste the following code:

```
import deepcopy
import math
import time
import threading

import numpy as np
import matplotlib.pyplot as plt

import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy
from geometry_msgs.msg import TwistStamped, Pose2D
from nav_msgs.msg import Odometry

from irobot_create_msgs.action import Undock
```

```
from tf_transformations import euler_from_quaternion

class Turtlebot(Node):

    def __init__(self, path, num_path, wf, ws):

        # Create and initialize the Node - this is required
        super().__init__('turtlebot_move')

        self.get_logger().info("Press Ctrl + C to terminate")

        # Current pose of the robot
        self.pose = Pose2D()

        # Logging counter
        self.logging_counter = 0

        # Store the robot trajectory
        self.trajectory = []

        # Store the ground truth trajectory
        self.gt_trajectory = []

        # Create publisher for publishing the velocity commands
        self.vel_pub = self.create_publisher(TwistStamped, "cmd_vel", 10)

        # Save passed parameters
        self.path = path
        self.wf = wf
        self.ws = ws
        self.num_path = num_path

        # Initial conditions
        self.previous_point = [0, 0]
        self.previous_velocity = [0, 0]
        self.vel_steering = 0.5
        self.vel = TwistStamped()

        # Create a subscriber for the ground truth
        self.odom_sub = self.create_subscription(Odometry,
            "odom",
            self.odom_callback,
            100)

        # Create a QoS profile for subscribing
        # to ground truth odometry published by Gazebo
        qos_profile = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,
            durability=DurabilityPolicy.VOLATILE,
            depth=10
        )

        # Logging counter for Ground Truth
        self.gt_logging_counter = 0

        # Create a subscriber for the ground truth
        # and use the profile previously created.
        self.gt_sub = self.create_subscription(Odometry,
            "sim_ground_truth_pose",
            self.gt_callback,
            qos_profile
            )

        # Undocking from the station
        self.undock_action_client = ActionClient(self, Undock, '/undock')

    def odom_callback(self, msg):
        """
```

```
        Callback function to get the robot odometry.

        @param msg - Message passed by the odom topic.
        """

        # get pose = (x, y, theta) from odometry topic
        quarternion = [msg.pose.pose.orientation.x,msg.pose.pose.orientation.y,\\
                    msg.pose.pose.orientation.z, msg.pose.pose.orientation.w]

        (roll, pitch, yaw) = euler_from_quaternion(quarternion)
        self.pose.theta = yaw
        self.pose.x = msg.pose.pose.position.x
        self.pose.y = msg.pose.pose.position.y

        # logging once every 100 times
        self.logging_counter += 1
        if self.logging_counter == 10:
            self.logging_counter = 0
            self.trajectory.append([self.pose.x, self.pose.y])  # save trajectory

            # display (x, y, theta) on the terminal
            self.get_logger().info("odom: x=" + str(self.pose.x) +\\
                ";  y=" + str(self.pose.y) + ";  theta=" + str(yaw))

    def smooth(self, path, alpha=1, weight_follow = 0.5, weight_smooth = 0.1, tolerance = 0.000001):
        """
        Smooth the trajectory here.

        @param path: Original path
        @param alpha: Description
        @param weight_follow: Description
        @param weight_smooth: Description
        @param tolerance: Description
        """

        # Make a copy of the original path
        newpath = deepcopy(path)

        # Large value
        change = float('inf')

        while change > tolerance:
            # Iterative update (repeat until convergence)
            change  = 0

            ############# TODO: Fill in your code here #######################

        # Return smoothed path
        return newpath

    def move_to_point(self, current_waypoint, next_waypoint):
        """
        Generate polynomial trajectory and move to current_waypoint.
        next_waypoint is to help determine the velocity to pass current_waypoint.

        @param current_waypoint: Current waypoint
        @param next_waypoint: Next waypoint
        """
        pass

    def gt_callback(self, msg):
        """
        Callback function for ground truth pose.

        @param msg - Message passed by Gazebo.
        """
        # get pose = (x, y, theta) from odometry topic
        quarternion = [msg.pose.pose.orientation.x,msg.pose.pose.orientation.y,\\
```

```
                    msg.pose.pose.orientation.z, msg.pose.pose.orientation.w]

        (roll, pitch, yaw) = euler_from_quaternion(quarternion)

        self.gt_logging_counter += 1
        if self.gt_logging_counter % 10 == 0:
            self.gt_logging_counter = 0
            self.gt_trajectory.append([msg.pose.pose.position.x, msg.pose.pose.position.y, yaw])
            self.get_logger().info(f"GT: ({msg.pose.pose.position.x:.3f}, \\
                                    {msg.pose.pose.position.y:.3f}, {yaw:.3f})")

    def run(self):
        """
        Run function.
        """

        self.get_logger().info('Running Gradient Descent...')

        # Move between points using gradient descent
        waypoints = self.path
        while True:

            if len(waypoints) == 1:
                break

            waypoints[0] = [self.pose.x, self.pose.y]

            # TODO: use gradient descent for the optimal trajectory
            # TODO: call self.wf and self.ws to get weight_follow and weight_smooth

            waypoints = self.move_to_point(waypoints[0], waypoints[1])

            # TODO: update the path to optimize the rest of the trajectory

    def save_trajectory(self):
        """
        Save trajectory to disk.
        """
        np.savetxt(f"trajectory_{self.num_path}.csv", np.array(self.trajectory), fmt='%f', delimiter
=',')
        np.savetxt(f"trajectory_ground_truth_{self.num_path}.csv", np.array(self.gt_trajectory), fmt
='%f', delimiter=',')

    def printpaths(self, path,newpath):
        """
        Print paths to the console. This will show the initial path before smoothing.

        @param path: Original path.
        @param newpath: New path.
        """

        for old, new in zip(path, newpath):
            self.get_logger().info(
                '['+ ', '.join('%.3f'%x for x in old) + \\
                '] -> ['+ ', '.join('%.3f'%x for x in new) +']')

    def plot(self, path, newpath):
        """
        Plotting function.

        @param path - Original path.
        @patam newpath - New path.
        """

        x =  [x for [x, y] in path]
        y =  [y for [ax, y] in path]

        # This plots the smooth path overlayed on the original
```

```
        plt.plot(x, y, color='r', marker='o')
        x2 =  [x for [x, y] in newpath]
        y2 =  [y for [ax, y] in newpath]
        plt.plot(x2, y2, color='b', marker='o')
        plt.xlim(-1, 3)
        plt.ylim(-1, 3)
        plt.show()

def gradient_descent_entry_point_function(args=None):
    """
    Entry point function for the Gradient Descent node.

    @param args - Arguments that need to be passed to ROS.
    """

    # Initialize the client library - this is required
    rclpy.init(args=args)

    #path1
    path1 = [[0,0], [0,1], [0,2], [0,3], [0,4], [1,4], [2,4], [2,3], [2,2], [3,2], [4,2], [4,1], [4,
0]]

    #path2
    path2 = [[0.5, 0], [0.5, -0.5], [1, -0.5], [1, 0], [1, 0.5],\\
                        [1.5, 0.5], [1.5, 0], [1.5, -0.5], [1, -0.5], [1, 0],\\
                        [1, 0.5], [0.5, 0.5], [0.5, 0], [0, 0], [0, 0]]

    # Create the publisher node
    turtlebot_node = Turtlebot(path1, 1, wf=0.1, ws=0.1)
    # turtlebot_node = Turtlebot(path2, 2, wf=0.1, ws=0.1)

    # Spin so that the node is busy processing
    # while no exceptions are encountered and ROS
    # is executing
    # For this lab - spin rclpy on separate thread
    thread = threading.Thread(target=rclpy.spin, args=(turtlebot_node,), daemon=True)
    thread.start()

    # Allow time for other nodes to start
    time.sleep(5)

    try:
        turtlebot_node.run()
    except KeyboardInterrupt:
        turtlebot_node.get_logger().info("Keyboard Interrupt.")
    finally:
        turtlebot_node.save_trajectory()

    # Destroy the node explicitly
    turtlebot_node.destroy_node()

    # Shut down this context
    rclpy.shutdown()

if __name__ == '__main__':
    gradient_descent_entry_point_function()
```

# Gradient Descent Path Smoothing

Let `P` denote a set of original points (returned by discrete planning algorithm, like `A*`). Let `T` be the set of points belonging to the optimized trajectory.

$$f_i = wf \cdot \frac{(p_i - t_i)^2}{2} + ws \cdot \left(\frac{t_{i-1} + t_{i+1}}{2} - t_i\right)^2$$

For each optimized $t_i$ point, define the cost function as:

- Path Following (error between original and optimized point)
- Smoothness term (distance between optimized point and the previous and next point)

# Sample Code for Polynomial Time Scaling

- Open a new terminal and go to your `eecs256aw25` package. We will start from a new python script.

```
cd ~/ros2_ws/src/eecs256aw25/eecs256aw25/
touch polynomial_timescaling.py
gedit polynomial_timescaling.py
```

- Please copy and paste the following code.

```
import deepcopy
import math
import time
import threading

import numpy as np
import matplotlib.pyplot as plt

import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy
from geometry_msgs.msg import TwistStamped, Pose2D
from nav_msgs.msg import Odometry

from irobot_create_msgs.action import Undock
from tf_transformations import euler_from_quaternion

class Turtlebot(Node):

    def __init__(self, path, num_path, wf, ws):

        # Create and initialize the Node - this is required
        super().__init__('turtlebot_move')

        self.get_logger().info("Press Ctrl + C to terminate")

        # Current pose of the robot
        self.pose = Pose2D()

        # Logging counter
        self.logging_counter = 0

        # Store the robot trajectory
        self.trajectory = []

        # Store the ground truth trajectory
        self.gt_trajectory = []

        # Create publisher for publishing the velocity commands
        self.vel_pub = self.create_publisher(TwistStamped, "cmd_vel", 10)

        # Save passed parameters
```

```
        self.path = path
        self.wf = wf
        self.ws = ws
        self.num_path = num_path

        # Create a subscriber for the ground truth
        self.odom_sub = self.create_subscription(Odometry,
            "odom",
            self.odom_callback,
            100)

        # Create a QoS profile for subscribing
        # to ground truth odometry published by Gazebo
        qos_profile = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,
            durability=DurabilityPolicy.VOLATILE,
            depth=10
        )

        # Logging counter for Ground Truth
        self.gt_logging_counter = 0

        # Create a subscriber for the ground truth
        # and use the profile previously created.
        self.gt_sub = self.create_subscription(Odometry,
            "sim_ground_truth_pose",
            self.gt_callback,
            qos_profile
            )

        # Undocking from the station
        self.undock_action_client = ActionClient(self, Undock, '/undock')

    def odom_callback(self, msg):
        """
        Callback function to get the robot odometry.

        @param msg - Message passed by the odom topic.
        """

        # get pose = (x, y, theta) from odometry topic
        quarternion = [msg.pose.pose.orientation.x,msg.pose.pose.orientation.y,\\
                    msg.pose.pose.orientation.z, msg.pose.pose.orientation.w]

        (roll, pitch, yaw) = euler_from_quaternion(quarternion)
        self.pose.theta = yaw
        self.pose.x = msg.pose.pose.position.x
        self.pose.y = msg.pose.pose.position.y

        # logging once every 100 times
        self.logging_counter += 1
        if self.logging_counter == 100:
            self.logging_counter = 0
            self.trajectory.append([self.pose.x, self.pose.y])  # save trajectory

            # display (x, y, theta) on the terminal
            self.get_logger().info("odom: x=" + str(self.pose.x) +\\
                ";  y=" + str(self.pose.y) + ";  theta=" + str(yaw))

    def polynomial_time_scaling_3rd_order(self, p_start, v_start, p_end, v_end, T):
        """
        Polynomial timescaling for 3rd order. Returns the matrix coefficients.

        @param p_start - Position of the start point.
        @param v_start - Velocity of the start point.
        @param p_end - Position of the end point.
        @param v_end - Velocity of the end point.
        @param T - the desired time to complete this segment of trajectory (in second).
```

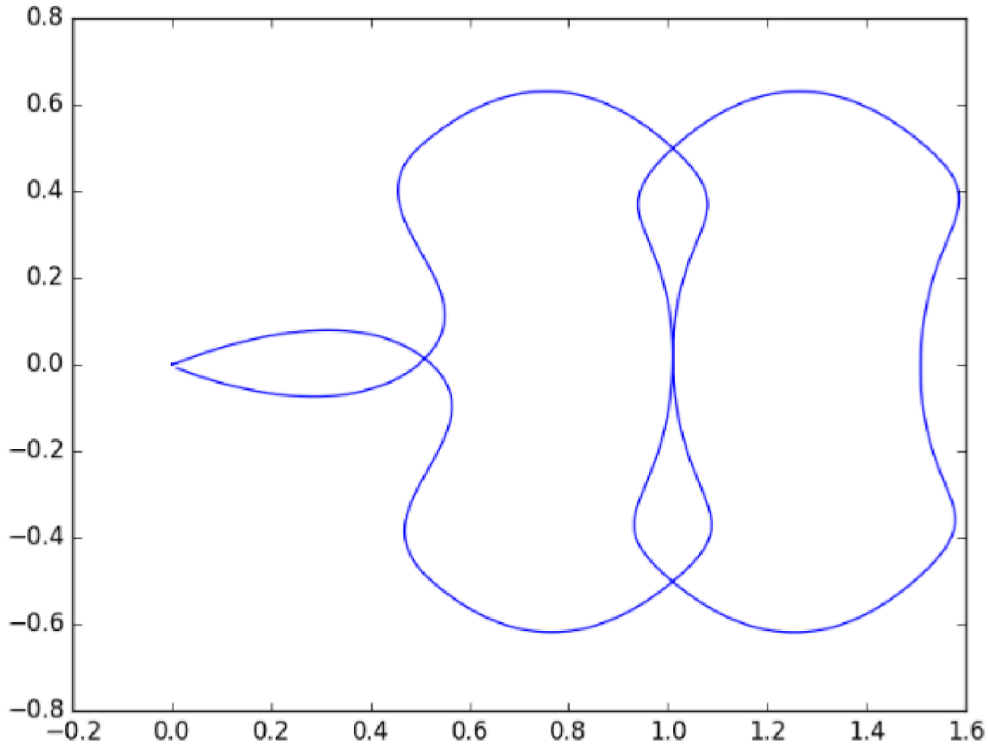
```
        @return the coefficients of this polynomial
        """
        pass

    def move_to_point(self, current_waypoint, next_waypoint):
        """
        Generate polynomial trajectory and move to current_waypoint.
        next_waypoint is to help determine the velocity to pass current_waypoint.

        @param current_waypoint: Current waypoint
        @param next_waypoint: Next waypoint
        """
        pass

    def gt_callback(self, msg):
        """
        Callback function for ground truth pose.

        @param msg - Message passed by Gazebo.
        """
        # get pose = (x, y, theta) from odometry topic
        quarternion = [msg.pose.pose.orientation.x,msg.pose.pose.orientation.y,\\
                    msg.pose.pose.orientation.z, msg.pose.pose.orientation.w]

        (roll, pitch, yaw) = euler_from_quaternion(quarternion)

        self.gt_logging_counter += 1
        if self.gt_logging_counter % 10 == 0:
            self.gt_logging_counter = 0
            self.gt_trajectory.append([msg.pose.pose.position.x, msg.pose.pose.position.y, yaw])
            self.get_logger().info(f"GT: ({msg.pose.pose.position.x:.3f}, \\
                                    {msg.pose.pose.position.y:.3f}, {yaw:.3f})")

    def run(self):
        """
        Run function.
        """

        self.get_logger().info('Running Polynomial Timescaling...')

        # Move between waypoints using polynomial timescaling
        waypoints = self.path
        for i in range(len(waypoints)-1):
            self.move_to_point(waypoints[i], waypoints[i+1])

    def save_trajectory(self):
        """
        Save trajectory to disk.
        """
        np.savetxt(f"trajectory_{self.num_path}.csv", np.array(self.trajectory), fmt='%f', delimiter
=',')
        np.savetxt(f"trajectory_ground_truth_{self.num_path}.csv", np.array(self.gt_trajectory), fmt
='%f', delimiter=',')

def polynomial_timescaling_entry_point_function(args=None):
    """
    Entry point function for the Polynomial Timescaling node.

    @param args - Arguments that need to be passed to ROS.
    """

    # Initialize the client library - this is required
    rclpy.init(args=args)

    #path1
    path1 = [[0,0], [0,1], [0,2], [0,3], [0,4], [1,4], [2,4], [2,3], [2,2], [3,2], [4,2], [4,1], [4,
0]]
```

```
    #path2
    path2 = [[0.5, 0], [0.5, -0.5], [1, -0.5], [1, 0], [1, 0.5],\\
                        [1.5, 0.5], [1.5, 0], [1.5, -0.5], [1, -0.5], [1, 0],\\
                        [1, 0.5], [0.5, 0.5], [0.5, 0], [0, 0], [0, 0]]

    # Create the publisher node
    turtlebot_node = Turtlebot(path1, 1, wf=0.1, ws=0.1)
    # turtlebot_node = Turtlebot(path2, 2, wf=0.1, ws=0.1)

    # Spin so that the node is busy processing
    # while no exceptions are encountered and ROS
    # is executing
    # For this lab - spin rclpy on separate thread
    thread = threading.Thread(target=rclpy.spin, args=(turtlebot_node,), daemon=True)
    thread.start()

    # Allow time for other nodes to start
    time.sleep(5)

    try:
        turtlebot_node.run()
    except KeyboardInterrupt:
        turtlebot_node.get_logger().info("Keyboard Interrupt.")
    finally:
        turtlebot_node.save_trajectory()

    # Destroy the node explicitly
    turtlebot_node.destroy_node()

    # Shut down this context
    rclpy.shutdown()

if __name__ == '__main__':
    polynomial_timescaling_entry_point_function()
```

- This script is following the same structure as the one used in Lab 3, except for the changes under run function.
- You need to complete the `move_to_point` function in this code, and make sure the robot can pass all the waypoints. The `polynomial_time_scaling_3rd_order` function is provided for your information only. You may or may not use it.

# Visualization

- You can reuse the visualization python script provided in Lab 3 to plot the trajectory. Remember to adjust the limit on x and y axes and **include the plot in the lab report**.
- An example of the trajectory is provided as follows. It is a bit overshooting. You can do better :)




# Polynomial Time Scaling

Suppose that we have figured out a path from point A to point B. The question is how fast the robot should follow the path (i.e. where it should be at each moment). This process of generating intermediate points (each associated with a time stamp) is called time scaling, or trajectory generation at large.

Recall that sending constant velocity commands from the very beginning would assume the robot can have infinite large acceleration at the first moment, which is impractical. Therefore, we would like to have a smooth trajectory that can best fit robot kinematics.

On the other hand, polynomial functions are smooth (infinitely differentiable) functions that can provide us smooth trajectories at all orders (position, velocity, acceleration, the derivative of acceleration, and so on). To this end, polynomial time scaling becomes a good choice to generate trajectories.

Suppose that we plan to move from point A to point B in `T` seconds by following a straight line. Then where the robot should be at each moment can be described by the following function. It provides the position of the robot `x(t)` at any given time `t`.

$$x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3, t \in [0, T]$$

Accordingly, the expected velocity at any time t can be described by the derivative of this polynomial function.

$$\dot{x}(t) = a_1 + 2a_2 t + 3a_3 t^2, t \in [0, T]$$

The key is to figure out the coefficients of this function **for each and every trajectory segment** and **for both x and y coordinates**. In other words, coefficients can vary from segment to segment, as each segment should adopt different initial and terminal conditions to best fit its needs. Fortunately, we can formulate this problem into a linear system of equation as follows, where:

$$x_0\,, x_T\,, \dot{x}_0\,, \dot{x}_T$$

are initial/terminal position/velocity respectively.

$$\begin{bmatrix} x_0 \\ x_T \\ \dot{x}_0 \\ \dot{x}_T \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ T^3 & T^2 & T & 1 \\ 0 & 0 & 1 & 0 \\ 3T^2 & 2T & 1 & 0 \end{bmatrix} \begin{bmatrix} a_3 \\ a_2 \\ a_1 \\ a_0 \end{bmatrix}$$

To solve this equation of the form x=Tα, we can simply take the advantage of the inverse matrix and have the solution $\alpha = T^{-1} x$. Once the coefficients are known, the position and the velocity at each moment can be obtained by evaluating the function x(t) and $\dot{x}(t)$ at t=0,0.1,0.2,...,T. (This is an example of running at 10Hz where the time interval is 0.1s.)

Finally, a PID controller (introduced in Lab 3) can be applied to track the desired position and velocity at each moment. To closely track the trajectory, the parameter Kp can take a larger value. As before, it is possible to only track the orientation by the PID controller and simply set the linear velocity to be the magnitude. Note that the orientation setpoint in this lab is changing all the time as the robot follows the trajectory (as opposed to a fixed setpoint in Lab 3).

So far we have introduced the basic steps to solve for a polynomial time scaling problem. The following are three final remarks regarding the selection of parameters.

1. Two ways to pick the time interval T (this is one of the drawbacks of this approach; you have to specify T ahead of time)
   - Fixed time interval for all segments.
   - Pick a preferred average speed, then determine T based on the distance to travel.
2. Notes on boundary conditions and continuity constraints
   - It is better to look at not only the current waypoint, but also the next one. Because normally the next waypoint can provide useful information to help determine how to pass the current waypoint.
   - For example, when moving from point A to point B by following a straight line, knowing that point C is to the right of point B is a good indicator to curve the current trajectory a bit more, such that this turning behavior can be evenly distributed in the trajectory and avoid a sharp turn at point B.
   - In practice, the magnitude of the velocity can be set to a preferred speed, and the direction of the velocity of passing point B can be set to the direction from point A to point C.
3. Discussion on the numerical stability of polynomial functions. It is possible to use a continuous timeline for all trajectories (i.e. $[0, T_1]$ for the first segment, $[T_1, T_2]$ for the second, and so on). However, this approach is not numerically stable, especially when the order of the polynomial is

higher. For example, in a 7th order polynomial function, as T grows larger, to make the term $\alpha_7 t^7$ reasonably small, the parameter $\alpha_7$ will have to be at the level of $10^{-10}$ or even smaller. Conclusion: we recommend using relative time scale [0, T] for all segments (i.e. reset timing every time).

# Ground Truth

- Ground truth is also provided. Please check the function `gt_callback` . **You should not use it for the robot pose.** Rather, you should use it to tune any parameters or check for deviation between the trajectories the robot follows (use it for comparing the distance threshold of `0.1` m).

# Submission

1. Submission: group submission via eLearn / Canvas.
2. **Demo: not required**
3. Due time: 11:59pm, February 25, Wednesday
4. Files to submit:
   - lab6_report.pdf (Include the plots of the trajectories and screenshots of the robot at 4 different positions for each path)
   - gradient_descent.py
   - polynomial_timescaling.py
5. Grading rubric:
   - 
     - 30% Clearly describe your approach and explain your code in the lab report and video.
   - 
     - 20% Plot the trajectories and discuss the results under different parameters.
   - 
     - 25% Pass the performance test on the Gradient Descent Path Smoothing Task.
   - 
     - 25% Pass all waypoints using 3rd order polynomial trajectories.
   - 10% Penalty applies for each late day. No submission will be accepted beyond four days from the due date.

# Grading

The scripts will be tested on a Ubuntu setup using a similar ROS + Gazebo environment.

Testing parameters are as follows.

1. **Gradient Descent**: How closely the smoothed path adheres to the original waypoints and measure the overall smoothness.
2. **Polynomial Timescaling:**
   - The tolerance for distance error is set to 0.1m. For example, passing point [3.96, 3.94] is approximately equivalent to passing point [4, 4].
   - The time limit is set to 400 seconds.